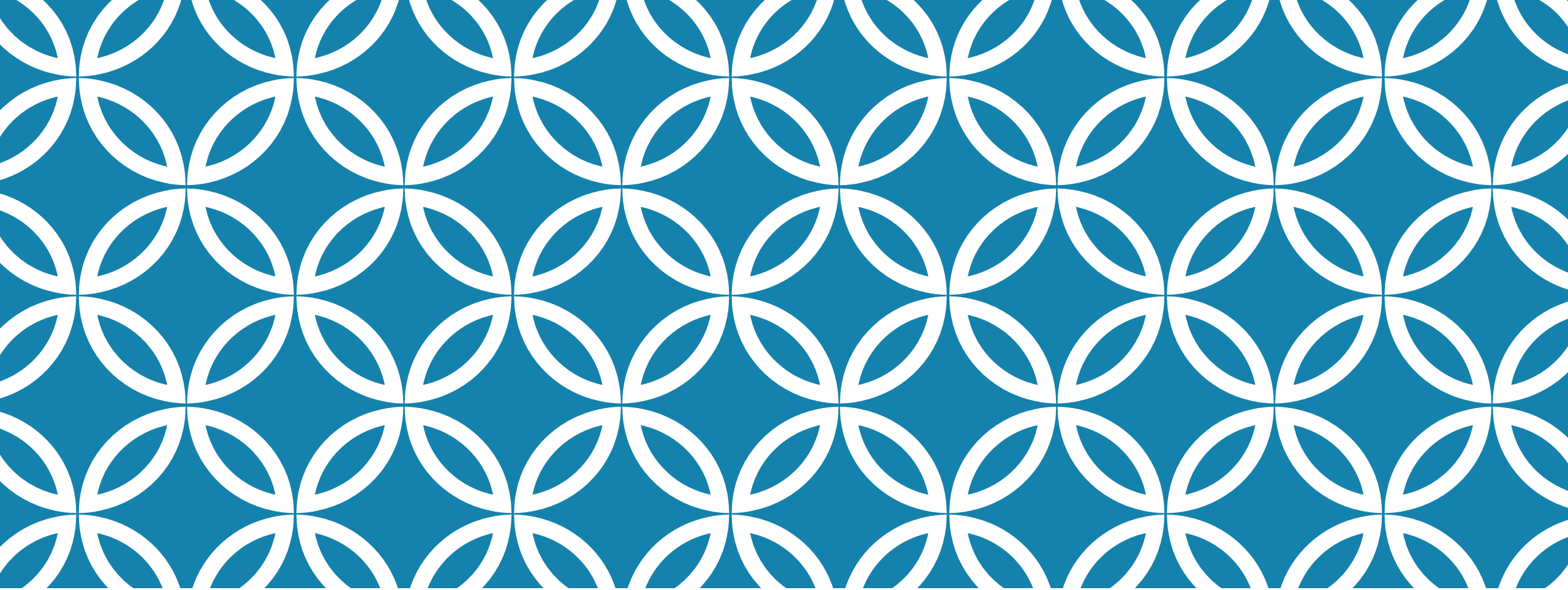


PARSERS

CLASSIFICATION OF GRAMMARS – 1/5

Grammars are classified based on the nature of productions used in them

Type-0 grammars

- Known as **phrase structure grammars**
- Contain productions of the form

$$\alpha ::= \beta$$

where both α and β can be strings of Ts and NTs

- Such productions permit arbitrary substitution of strings during derivation or reduction
- Hence they are **not relevant** to specification of **programming languages**

CLASSIFICATION OF GRAMMARS – 2/5

- Type-1 grammars
  - Known as **context sensitive grammars**
  - Production specify that derivation or reduction of strings can take place only in specific contexts
  - A Type-1 production has the form

    $$\alpha\ A\ \beta ::= \alpha\ \pi\ \beta$$

  - Thus a string π in a sentential form can be replaced by 'A' (or vice versa) only when it is enclosed by the strings α and β
  - These grammars are also **not** particularly **relevant for PL** specification since recognition of PL constructs is not context sensitive in nature

CLASSIFICATION OF GRAMMARS – 3/5

Type-2 Grammars

- Impose no context requirements on derivations or reductions
- It has the form

  $A :: = \pi$

  which can be applied independent of its context
- Known as **context free grammars (CFG)**
- CFGs are **ideally suited for programming language** specification

CLASSIFICATION OF GRAMMARS – 4/5

- Type-3 Grammars
  - Characterized by the production

    A ::= tB | t or

    A ::= t | tB
  - These productions also satisfy Type-2 grammars
  - Use of Type-3 specification is restricted to specification of lexical units, eg. Identifiers, constants, labels, etc

CLASSIFICATION OF GRAMMARS – 5/5

- This can be clearly seen when we rewrite the production for <id> in the form Bt | t, as

  <id> ::= l | <id> l | <id> d

  where l and d stands for letter and digit resp.

- They are also known as linear grammars or **regular grammars**
- Further categorized into left-linear and right-linear grammars depending on whether NT in RHS alternative appears at extreme left or extreme right

CONTEXT FREE GRAMMARS (CFGS)

Four tuples for a grammar G

$$G = (V, T, P, S)$$

V : set of non terminals

T : set of terminals

P : set of productions

S : start symbol

S → 0 A 1

A → 0 | 1

Start symbol → S

Non terminals → S, A

Terminals → 0, 1

Number of Productions → 3

DERIVATION AND REDUCTION

A grammar G is used for **two purposes**

- To **generate valid strings** of L_G → **derivation**
- To **'recognize' valid strings** of L → **reduction**

A parse tree is used to depict the syntactic structure of a valid string as it emerges during a sequence of derivations or reductions

DERIVATION - 1/6

Let production P_1 of grammar G be of the form

$$P_1 : A ::= \alpha$$

and let β be a string such that $\beta \equiv \gamma A \theta$, then replacement of A by α in string β constitutes a derivation according to P_1

i.e. $\beta \equiv \gamma \alpha \theta$

DERIVATION - 2/6

Use the notation $N \Rightarrow \eta$ to denote direct derivation of η from N

Notation $N \Rightarrow^* \eta$ to denote transitive derivation of η (i.e. derivation in zero or more steps) from N

Thus, $A \Rightarrow \alpha$ only if $A ::= \alpha$ is a production of G and $A \Rightarrow \delta$ if $A \Rightarrow \ldots. \Rightarrow^* \delta$

DERIVATION - 3/6

We can use this notation to define a valid string according to a grammar G as follows:

δ is a valid string according to G only if

$S \Rightarrow^* \delta$, where S is distinguished symbol of G

DERIVATION - 4/6

Eg.: Derivation of the string the boy according to grammar can be depicted as

<Noun Phrase> $\Rightarrow$ <Article> <Noun>

$\Rightarrow$ the <Noun>

$\Rightarrow$ the boy

A string α such that S $\Rightarrow^*$ α is a sentential form of L_G

The string α is a sentence of L_G if it consists of only Ts

DERIVATION - 5/6

Consider the grammar

<Sentence> ::= <Noun Phrase> <Verb Phrase>

<Noun Phrase> ::= <Article> <Noun>

<Verb Phrase> ::= <Verb> <Noun Phrase>

<Article> ::= a | an | the

<Noun> ::= boy | apple

<Verb> ::= ate

DERIVATION - 6/6

The following strings are sentential forms of L_G

<Noun Phrase> <Verb Phrase>

the boy <Verb Phrase>

<Noun Phrase> ate <Noun Phrase>

the boy ate <Noun Phrase>

the boy ate an apple

However, only **the boy ate an apple** is a sentence

REDUCTION - 1/3

Let production P_1 of grammar G be of the form

$$P_1 : A ::= \alpha$$

and let σ be a string s.t. $\sigma \equiv \gamma\alpha\theta$, then replacement of α by A in string σ constitutes a reduction according to production P_1

Use the notations $\eta \rightarrow N$ and $\eta \rightarrow^* N$ to depict direct and transitive reduction, respectively

Thus, $\alpha \rightarrow A$ only if $A ::= \alpha$ is a production of G and $\alpha \rightarrow A$ if $\alpha \rightarrow \ldots \rightarrow^* A$

REDUCTION - 2/3

We define the validity of some string δ according to grammar G as follows :

δ is a valid string of L_G if $\delta \rightarrow^* S$, where S is distinguished symbol of G

REDUCTION - 3/3

Step	String
0	the boy ate an apple
1	<Article> boy ate an apple
2	<Article> <Noun> ate an apple
3	<Article> <Noun> <Verb> an apple
4	<Article> <Noun> <Verb> <Article> apple
5	<Article> <Noun> <Verb> <Article> <Noun>
6	<Noun Phrase> <Verb> <Article> <Noun>
7	<Noun Phrase> <Verb> <Noun Phrase>
8	<Noun Phrase> <Verb Phrase>
9	<Sentence>

the boy ate an apple → *<Sentence>

Eg.: To determine the validity of string

the boy ate an apple

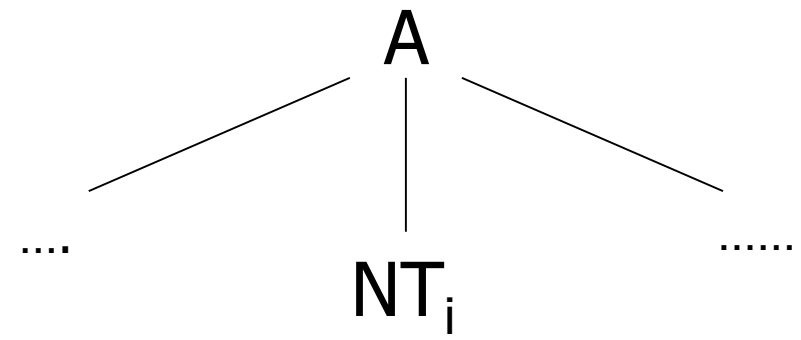
According to grammar we perform the following reductions

Grammar

<Sentence> ::= <Noun Phrase> <Verb Phrase>
<Noun Phrase> ::= <Article> <Noun>
<Verb Phrase> ::= <Verb> <Noun Phrase>
<Article> ::= a | an | the
<Noun> ::= boy | apple
<Verb> ::= ate

PARSE TREES – 1/3

- A sequence of derivations or reductions reveals the syntactic structure of a string w.r.t. G
- We depict the syntactic structure in the form of a parse tree
- Derivation according to production A ::= α gives rise to following elemental parse tree

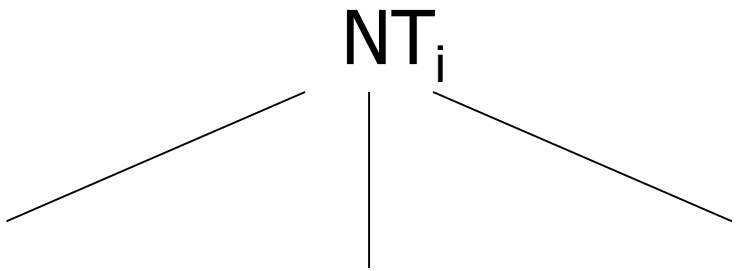


(Sequence of Ts and NTs constituting α)

PARSE TREES – 2/3

A subsequent step in the derivation replaces an NT in α, say NT_i, by a string

We can build another elemental parse tree to depict this derivation



PARSE TREES – 3/3

We can combine the two trees by replacing the node of NT_i in first tree by this tree

The parse tree has grown in downward direction due to derivation

Obtain a parse tree from a sequence of reductions by performing the converse actions

Such a tree would grow in **upward direction**

PARSERS

A parser for a grammar is a program that takes input a string w and produces output either a parse tree for w, if valid, or an error if not valid

Two types

- Top down parser
- Bottom up parser

SHIFT REDUCE PARSER

It is a **bottom up parser**

Constructs a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top)

Reduce a string to the start symbol of the grammar

$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

Rightmost Derivation:

$S \Rightarrow aA\mathbf{B}e \Rightarrow a\mathbf{A}de \Rightarrow a\mathbf{A}bcde \Rightarrow abbcde$

abbcde

aAbcde

aAde

aABe

S

Handle – it is a substring that matches the RHS of some production AND whose reduction to the non-terminal on the LHS is a step along the reverse of some rightmost derivation

$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

abbcde → aAbcde → aAde → aABe → S

SHIFT REDUCE PARSING (WITH A STACK)

Two problems

- locate a handle
- decide which production to use

Using stack

- **Shift** -- input symbols into the stack until a handle is found on top of it.
- **Reduce** -- the handle to the corresponding non-terminal.
- **Accept** -- when the input is consumed and only the start symbol is on the stack,
- **Error** -- when it does not lead to the start symbol

$E \rightarrow \quad E + E \mid \ E * E \mid (E) \mid id$

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift

$E \rightarrow \quad E + E \mid \ E * E \mid (E) \mid id$

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id

$E \rightarrow \quad E + E \mid \ E * E \mid (E) \mid id$

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift

$E \rightarrow \quad E + E \mid \ E * E \mid (E) \mid id$

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift

E → E + E | E * E | (E) | id

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id

E → E + E | E * E | (E) | id

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
	id$	Shift

$E \rightarrow \quad E + E \mid E * E \mid (E) \mid id$

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
	id$	Shift

E → E + E | E * E | (E) | id

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
$E *	id$	Shift
	$	Reduce E → id

E → E + E | E * E | (E) | id

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
$E *	id$	Shift
$E * id	$	Reduce E → id
	$	Reduce E→ E * E

E → E + E | E * E | (E) | id

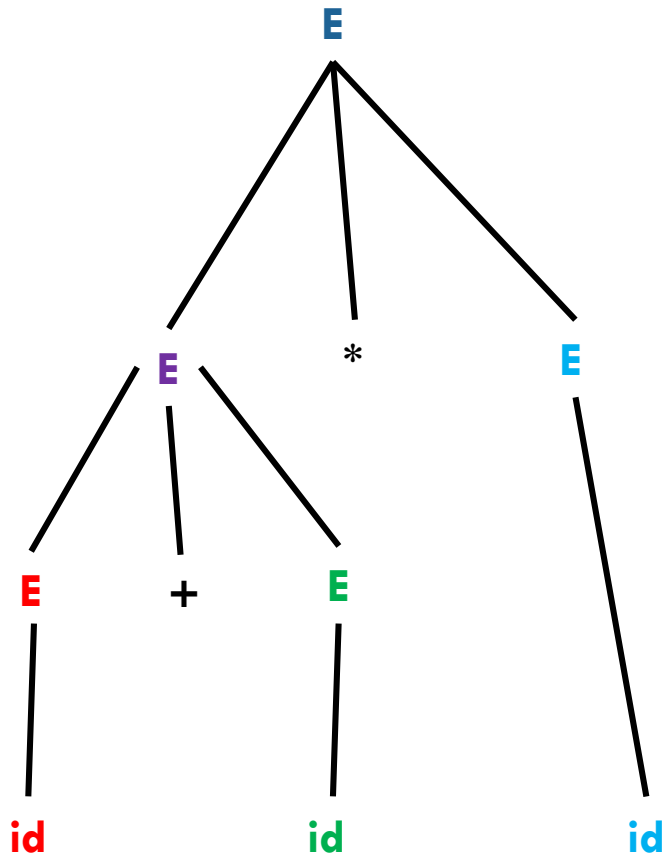
Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
$E *	id$	Shift
$E * id	$	Reduce E → id
$E * E	$	Reduce E→ E * E
	$	Accept

E → E + E | E * E | (E) | id

Reduce the string id + id * id using the above grammar.

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E+id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
$E *	id$	Shift
$E * id	$	Reduce E → id
$E * E	$	Reduce E→ E * E
$E	$	Accept
$		



Start with this input

STACK	INPUT	Remark
$	id + id * id$	Shift
$ id	+ id * id$	Reduce by E → id
$E	+ id * id$	Shift
$E+	id * id$	Shift
$E + id	* id $	Reduce E → id
$E + E	* id $	Reduce E→ E + E
$E	id$	Shift
$E *	id$	Shift
$E * id	$	Reduce E → id
$E * E	$	Reduce E→ E * E
$E	$	Accept
$		

Draw the parse tree starting from the first operation in the stack

EXERCISES

E → E + E | E * E | (E) | id

1) Check whether the string **(id + id) * id is recognized** using the above grammar

2) Reduce the string **id * (id + id)** using the above grammar